PRO2000 - PROGRAMMING

Week 42- Classes, objects, constructors

Gebremariam Assres
Gebremariam.assres@kristiania.no

Today's topics

- ADTs
- Classes and objects
- Member functions, variables
- Constructors, destructors
- Passing and returning objects
- Object-oriented design

Abstract Data Types

- Abstract data type (ADT) are programmer-created data types.
- A user of ADT does not need to know implementation details
  - e.g., how data is stored or how operations on it are carried out
- Data Type defines the kind of values that can be stored and the operations that can be performed on it

Abstract Data Types ...

- Abstraction captures general info without details
  - e.g., an abstract triangle is a 3-sided polygon.
  - A specific triangle may be scalene, isosceles, or equilateral

- Procedural programming uses variables to store data and focuses on functions that occur in a program.
  - Data and functions are separate and distinct.

- Object-oriented programming is based on objects encapsulating data and functions that operate on it.

Classes and objects

- An object is software entity that combines data and functions that act on the data in a single unit:
  - attributes- the data items of an object, stored in member variables
  - member functions (methods)- functions that act on the attributes

- Data hiding is restricting access to certain members of an object:
  - allowing only member functions to directly access and modify the object's data

- Encapsulation is the bundling of an object's data and procedures into a single entity

Classes and objects ...

- Example of square object's data item, and functions:

Square

Member variables (attributes)

```
int side;
```

Member functions

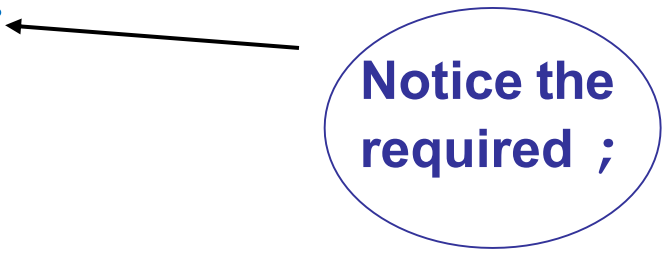
```
void setSide(int s)
{  side = s;    }

int getSide()
{  return side; }
```

Classes and objects ...

- **Class** is an ADT used to define objects in pattern
  - e.g., **string fName, lName;** creates two objects of the **string** class
- Class declaration format:

```
class className
{
    declaration;
    declaration;
};
```



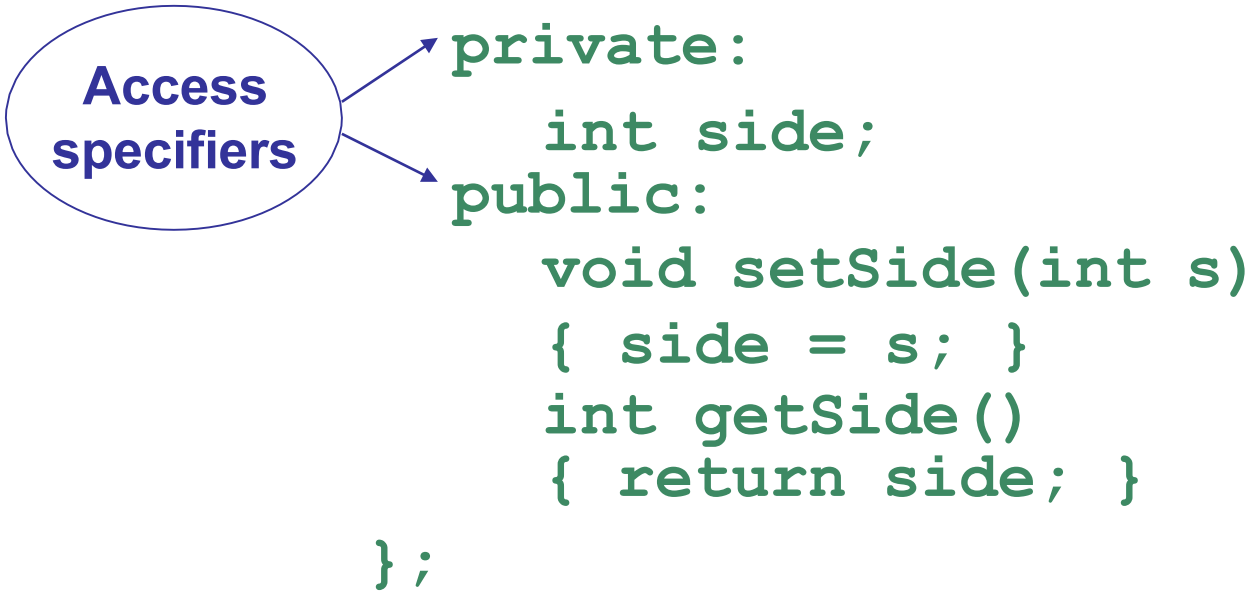
Classes and objects ...

- Access specifiers are used to control access to members of the class.
- Each member is declared to be either:
  - public:can be accessed by functions outside of the class or
  - private:can only be called by or accessed by functions that are members of the class

- Access specifiers can be listed in any order in a class, can appear multiple times, and if not specified, the default is private

Classes and objects ...

- Class Example



Classes and objects ...

- An object is an instance of a class- the creation of objects is defined just like other variables Square sq1, sq2;
- It can access members using dot operator
  sq1.setSide(5); cout << sq1.getSide();
- Objects can also be used as array elements (array of objects)
  This code creates array of 10 Square Square shapes[10]; objects

Classes and objects ...

- Like an array of structures, use an array subscript to access a specific object in the array
- Then use dot operator to access object's member methods

```
for (i = 0; i < 10; i++) cout << shapes[i].getSide() << endl;
```

- Initializing arrays of objects can use:
  - default constructor to perform same initialization for all objects
  - initialization list to supply specific initial values for each object

```
Square shapes[5] = {1,2,3,4,5};
```

  - If an object is initialized with a constructor that takes > 1 argument, the initialization list must include a call to constructor for that object:

```
Rectangle spaces[3] = { Rectangle(2,5), Rectangle(1,3), Rectangle(7,7)};
```

Member functions and variables

- Types of member functions:
  - Acessor, get, getter function uses but does not modify a member variable (e.g., getSide)
  - Mutator, set, setter function modify member variable (e.g., setSide)
- Member functions are part of a class declaration can place:
  - entire function definition inside class declaration- inline function
  - Or just the prototype inside the class declaration and write the function definition after the class
  - Only very short functions, like the one below, should be inline functions int getSide() { return side; }
  - A private member function is used for internal processing and can only be called by another member function of the same class.

Member functions and variables …

- Defining member functions after class declaration:
  - Put a function prototype in the class declaration
  - In the function definition, precede the function name with the class name and scope resolution operator (`::`) `int Square::getSide(){return side;}`
- Conventions:
  - Member variables are usually private while functions are usually public
  - Use get for naming accessor functions, set for mutator functions
  - A regular function call involves overhead, such as storing the return address and allocating memory for local variables.
  - An inline function avoids this by replacing the call with its code during compilation, potentially improving speed but increasing executable size.

Constructors and descructors

- A **constructor** is a member function that is often used to initialize data members of a class
  - Is called automatically when an object of the class is created
  - It must be a public member function
  - It must be named the same as the class
  - It must have no return type

Inline:

```
class Square{
      . . .
      public: Square(int
      s)
      { side = s; }
      . . .
};
```

Declaration outside the class:

```
Square(int);
//prototype
//in class

Square::Square(int s)
{ side = s;}
```

Constructors and descructors ...

- Overloading constructors
  - A class can have more than 1 constructor
  - Overloaded constructors in a class must have different parameter lists Square(); Square(int);
- Default constructor
  - Constructors can have any number of parameters.
  - A **default constructor** is one that takes no arguments either due to no parameters or all parameters have default values
  - If a class has any programmer-defined constructors, it must have a programmer-defined default constructor

Constructors and descructors ...

- Example of default constructor

```
class Square
{
private: int side;

public: Square()
    { side = 1; }
        // Other member
        // functions go here
};
```

Has no parameters

```
class Square
{
    private:
        int side;
    public:
        Square(int s = 1)
        { side = s; }
        // Other member
        // functions go here
};
```

Has parameter but it has a default value

Constructors and descructors ...

- Invoking a constructor:
  - To create an object using the default constructor, use no argument list and no () Square square1;
  - To create an object using a constructor that has parameters, include an argument list Square square1(8);
- Destructors
  - Is a public member function automatically called when an object is destroyed
  - The destructor name is ~*className*, *e.g.*, ~Square
  - It has no return type, takes no arguments
  - Only 1 destructor is allowed per class (*i.e.*, it cannot be overloaded)

Passing and returning objects

- Passing an object
  - A class object can be passed as an argument to a function
  - When passed by value, function makes a local copy of object, and the object in calling environment is unaffected by actions in function
  - When passed by reference, function can use set functions to modify the object.
  - Passing by value can slow down a program and waste space while by reference speeds up program, but function modifies parameter.
  - To save space and time, while protecting data change, use a const reference parameter void showData(const Square &s)

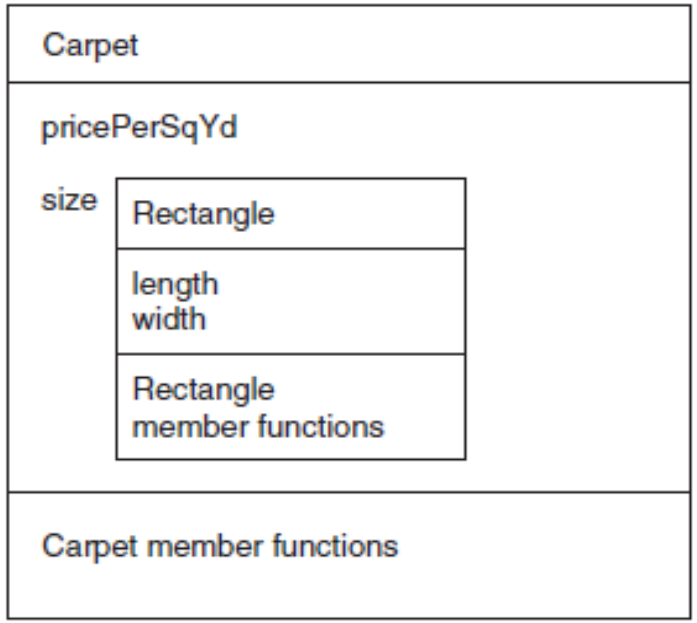
Passing and returning objects …

- Returning an object
  - A function can return an object prototype- Square initSquare () and a call s1 = initSquare();
  - The function must define a object for internal use and to use with return statement

```
Square initSquare()
{
   Square s;// local variable int inputSize;
   cout << "Enter the length of side: ";
   cin >> inputSize;
   s.setSide(inputSize); return
   s;
}
```

Object-oriented design

- Object composition
  - An object is a member variable of another object - used to design complex objects whose members are simpler objects
  - e.g., define a rectangle class. Then, define a carpet class and use a rectangle object as a member of a carpet object.



Object-oriented design ...

- Placing into separate files is considered good design- class declaration, member function definitions and using the class.
- Using Separate Files
  - Place class declaration in a header file that serves as the class specification file. Name the file classname.h (for example, Square.h)
  - Place member function definitions in a class implementation file. Name the file classname.cpp (for example, Square.cpp)
  - A client that uses the class must #include the class specification file and be compiled and linked with the class implementation file.
  - Use header guards (#ifndef, #define and #endif) in the .h file to prevent a header file from being included more than once.

Object-oriented design ...

- Object-oriented analysis is the a process of determining the functionality of a program from the requirements and includes
  - identification of objects and classes
  - definition of each class's attributes
  - identification of each class's behaviors
  - definition of the relationship between classes
- Identify objects and classes
  - Consider data elements and operations on these elements
  - Candidates include:
    - user-interface components (menus, text boxes, *etc.*),
    - I/O devices, physical objects,
    - historical data (employee records, transaction logs, *etc.*), and
    - the roles of human participants

Object-oriented design ...

- Define class attributes
  - Attributes are the data elements of an object of the class
  - They are necessary for the object to work in its role in the program
- Define class behaviors
  - For each class, identify what an object of a class should do
  - The behaviors determine some of the member functions of the class
- Possible relationships between classes:
  - Access ("uses-a"), ownership/Composition ("has-a"), and inheritance ("is-a")

Object-oriented design ...

- Technique for finding the classes:
  - Write a description of the problem domain (objects, events, etc. related to the problem). List the nouns, noun phrases, and pronouns. These are all candidate objects
  - Refine the list to include only relevant objects to the problem
- Determine class responsibilities:
  - What is the class responsible to know? What is the class responsible to do? Use these to define some of the member functions
- Object reuse
  - Object reuse is among the goals of object-oriented programming.
  - A class can be used to create objects in multiple programs thereby shortening development time

Up next: Exercises

- Now: 2 hours (12:15 to 14:00) of exercises (found on Canvas).

- Short exercise summary:
  - *Write, compile, and debug a C++ programs with emphasis on classes, objects and constructors/descructors.*
- Remember:
  - You should do approximately 4 more hours of the course (in a study group or alone) *before* our next session together. :-)